



Universidade Federal do Ceará
Campus Quixadá

QXD0011 - Fundamentos de Banco de Dados
Prof. Francisco Victor da Silva Pinheiro

Trabalho Final

Entrega 4 – Visões, Controle de Acesso e Aplicação Final

Quixadá-CE
2026

1. Integrantes da Equipe

Matrícula	Nome	E-mail
554930	Francisca Ariane dos Santos da Silva	francisca.silva@alu.ufc.br
493659	Gabriel Braga Martins	gabrielbragamartins@alu.ufc.br
539730	Pablo Brandão Passos	pablobrandao@alu.ufc.br
596042	Raul Camurça Rabelo de Almeida	raulcamurca@alu.ufc.br

2. Resumo da Proposta

Nesta quarta etapa do projeto UniCarona, o foco principal concentrou-se na evolução da camada de segurança, na simplificação do acesso aos dados dentro do SGBD PostgreSQL e no estabelecimento da arquitetura de integração com a aplicação externa desenvolvida em Python com o framework FastAPI.

Para alcançar esses objetivos, implementamos visões (views) complexas que encapsulam junções de múltiplas tabelas, reduzindo a complexidade de consultas repetitivas para a aplicação. Adicionalmente, estruturamos uma política estrita de controle de acesso (DCL), isolando os privilégios administrativos das permissões de consumo comum.

3. Visão no Banco de Dados

- **Descrição da Visão Criada (1)**

- 1. **VISÃO:** `vw_detalhes_caronas`

Esta visão consolida os dados operacionais de todas as caronas, trazendo os nomes do motorista, modelo do veículo, placa, origem, destino, data, horário e vagas disponíveis. A sua utilidade no contexto do sistema é servir como o endpoint principal de listagem de viagens da aplicação FastAPI. Ela encapsula cinco junções (JOINS) diretamente no motor do SGBD, evitando que a aplicação precise processar e enviar consultas SQL massivas e redundantes a cada requisição de busca, otimizando drasticamente o tráfego de rede e o desempenho da API.

- **Comando SQL da Visão (1)**

```
CREATE OR REPLACE VIEW vw_detalhes_caronas AS
SELECT C.id_carona,
       U.nome AS motorista,
       V.modelo AS veiculo,
       V.placa,
       Ll.nome_local AS origem,
```

```

        L2.nome_local AS destino,
        C.data,
        C.horario,
        C.vagas_disponiveis
FROM CARONA C
JOIN MOTORISTA M ON C.cpf_motorista = M.cpf
JOIN USUARIO U ON M.cpf = U.cpf
JOIN VEICULO V ON C.placa_veiculo = V.placa
JOIN LOCAL L1 ON C.id_origem = L1.id_local
JOIN LOCAL L2 ON C.id_destino = L2.id_local;

```

- **Descrição da Visão Criada (2)**

2. VISÃO: vw_ranking_motoristas

Esta visão agrega as notas da tabela de avaliações para gerar um ranking de reputação dos motoristas com média calculada e arredondada. A sua utilidade no contexto do sistema é fornecer uma camada automatizada de inteligência de negócio e segurança para a plataforma. Em vez de a aplicação recalculer a média de notas de um condutor sob demanda toda vez que um aluno buscar uma carona, a visão entrega o dado estatístico pronto e ordenado. Isso permite criar filtros automáticos no aplicativo para destacar motoristas confiáveis e monitorar perfis que fiquem abaixo da média aceitável da comunidade universitária.

- **Comando SQL da Visão (2)**

```

CREATE OR REPLACE VIEW vw_ranking_motoristas AS
SELECT U.nome AS motorista,
       ROUND(AVG(A.nota)::numeric, 2) AS media_reputacao,
       COUNT(A.id_avaliacao) AS total_avaliacoes_recebidas
FROM AVALIACAO A
JOIN MOTORISTA M ON A.cpf_avaliado = M.cpf
JOIN USUARIO U ON M.cpf = U.cpf
GROUP BY U.cpf, U.nome
ORDER BY media_reputacao DESC;

```

4. Criação de Usuários e Controle de Acesso

- **Usuários Criados**

unicarona_admin: Papel administrativo criado com privilégios totais de leitura e escrita (GRANT ALL PRIVILEGES) sobre a base de dados, esquemas, tabelas e sequências do sistema UniCarona. Este usuário simula as credenciais utilizadas por serviços de background ou administradores do sistema para realizar manutenções, manipulações estruturais (DDL) e limpezas de logs.

unicarona_leitura: Papel com restrições severas de gravação, atuando sob a política de privilégio mínimo. Ele possui permissões estritas de conexão e consulta (SELECT) nas tabelas e visões do esquema público, tendo qualquer

tentativa de modificação, inserção ou deleção de registros previamente revogada (REVOKE) diretamente no núcleo do SGBD.

- **Comandos SQL Utilizados**

```
-- Criando os usuários
CREATE USER unicarona_admin WITH PASSWORD 'admin_unicarona_2026';
CREATE USER unicarona_leitura WITH PASSWORD 'leitura_unicarona_2026';
-- 1. Permissões do Usuário Admin (Privilégios totais de leitura e escrita)
GRANT ALL PRIVILEGES ON DATABASE unicarona TO unicarona_admin;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO unicarona_admin;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO unicarona_admin;
-- 2. Permissões do Usuário Somente Leitura (Apenas SELECT)
-- Revogando qualquer direito de escrita
REVOKE INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public FROM
unicarona_leitura;
-- Concedendo estritamente o acesso de leitura nas tabelas e visões
GRANT CONNECT ON DATABASE unicarona TO unicarona_leitura;
GRANT USAGE ON SCHEMA public TO unicarona_leitura;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO unicarona_leitura;
GRANT SELECT ON ALL SEQUENCES IN SCHEMA public TO unicarona_leitura;
```

5. Aplicação Desenvolvida

- **Linguagem/Ferramenta Utilizada:**

A aplicação foi desenvolvida utilizando a linguagem Python 3, empregando o framework FastAPI para a construção rápida e documentada da API RESTful. Para a manipulação e mapeamento objeto-relacional (ORM), utilizamos a biblioteca SQLAlchemy, juntamente com o Pydantic para validação dos dados de entrada e saída. A comunicação com o PostgreSQL foi garantida pelo driver psycopg2.

- **Funcionalidades Implementadas:**

O foco da aplicação foi implementar o CRUD (Create, Read, Update, Delete) central do nosso sistema: o gerenciamento da tabela CARONA. As seguintes funcionalidades (rotas) foram disponibilizadas e testadas via interface do Swagger UI:

Create (POST [/caronas/](#)): Permite inserir uma nova oferta de carona no sistema, vinculando motorista, veículo, locais de origem/destino e vagas.

Read (GET [/caronas/](#) e [/caronas/{id}](#)): Permite listar todas as caronas disponíveis ou buscar os detalhes de uma viagem específica pelo seu identificador.

Update (PUT [/caronas/{id}](#)): Permite atualizar informações de uma carona existente (ex: diminuir a quantidade de vagas disponíveis após uma reserva).

Delete (DELETE [/caronas/{id}](#)): Permite cancelar/remover uma carona do banco de dados.

- **Testes da API:**

Os testes foram realizados utilizando dois perfis de acesso. Com o usuário ***unicarona_admin***, todas as operações CRUD foram executadas com sucesso. Ao alterar a conexão para o usuário ***unicarona_leitura***, apenas as operações de consulta (GET) permaneceram disponíveis. As tentativas de inserção (POST), atualização (PUT) e remoção (DELETE) resultaram em erro na API, pois o PostgreSQL bloqueou as operações de escrita devido às permissões restritas do usuário, comprovando o funcionamento do controle de acesso.

- **Prints ou Capturas de Tela da Aplicação:**

GET:

```
Request URL
http://127.0.0.1:8000/caronas/?skip=0&limit=100

Server response
Code    Details
200

Response body
[
  {
    "cpf_motorista": "00000000011",
    "placa_veiculo": "ABC1A11",
    "id_origem": 4,
    "id_destino": 1,
    "data": "2026-07-12",
    "horario": "18:00:00",
    "vagas_disponiveis": 3,
    "id_carona": 32
  },
  {
    "cpf_motorista": "00000000011",
    "placa_veiculo": "ABC1A11",
    "id_origem": 1,
    "id_destino": 5,
    "data": "2026-07-13",
    "horario": "06:30:00",
    "vagas_disponiveis": 2,
    "id_carona": 33
  },
  {
    "cpf_motorista": "00000000012",
    "placa_veiculo": "ABC1A12",
    "id_origem": 1,
    "id_destino": 4,
    "data": "2026-07-13",
    "horario": "06:30:00",
    "vagas_disponiveis": 2,
    "id_carona": 34
  }
]
```

GET POR ID:

```
Execute

Responses

Curl
curl -X 'GET' \
  'http://127.0.0.1:8000/caronas/3' \
  -H 'accept: application/json'

Request URL
http://127.0.0.1:8000/caronas/3

Server response
Code    Details
200

Response body
{
  "cpf_motorista": "000000000133",
  "placa_veiculo": "0XX1043",
  "id_origem": 3,
  "id_destino": 8,
  "data": "2026-06-21",
  "horario": "10:00:00",
  "vagas_disponiveis": 4,
  "id_carona": 3
}
```

POST:

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/caronas/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "cpf_motorista": "00000000011",
    "placa_veiculo": "ABC1A11",
    "id_origem": 2,
    "id_destino": 1,
    "data": "2026-07-03",
    "horario": "09:46:51.751Z",
    "vagas_disponiveis": 2,
    "id_carona": 999
  }'
```

Request URL

http://127.0.0.1:8000/caronas/

Server response

Code	Details
200	Response body <pre>{ "cpf_motorista": "00000000011", "placa_veiculo": "ABC1A11", "id_origem": 2, "id_destino": 1, "data": "2026-07-03", "horario": "09:46:51.751000", "vagas_disponiveis": 2, "id_carona": 999 }</pre>

PUT:

Responses

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/caronas/4' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "vagas_disponiveis": 3
  }'
```

Request URL

http://127.0.0.1:8000/caronas/4

Server response

Code	Details
200	Response body <pre>{ "cpf_motorista": "00000000134", "placa_veiculo": "OXX1044", "id_origem": 4, "id_destino": 9, "data": "2026-06-21", "horario": "11:00:00", "vagas_disponiveis": 3, "id_carona": 4 }</pre>

DELETE:

Carona_id	999
integer (path)	
Execute	
Responses	
Curl	
<pre>curl -X 'DELETE' \ 'http://127.0.0.1:8000/caronas/999' \ -H 'accept: application/json'</pre>	
Request URL	
<pre>http://127.0.0.1:8000/caronas/999</pre>	
Server response	
Code	Details
200	Response body
	<pre>{ "mensagem": "Carona cancelada/deletada com sucesso" }</pre>

Teste com Controle de Acesso somente leitura:

POST:

Curl	
<pre>curl -X 'POST' \ 'http://127.0.0.1:8000/caronas/' \ -H 'accept: application/json' \ -H 'Content-Type: application/json' \ -d '{ "cpf_motorista": "00000000011", "placa_veiculo": "ABC1A11", "id_origem": 2, "id_destino": 3, "data": "2026-07-03", "horario": "09:46:51.751Z", "vagas_disponiveis": 2, "id_carona": 999 }'</pre>	
Request URL	
<pre>http://127.0.0.1:8000/caronas/</pre>	
Server response	
Code	Details
500 <i>Undocumented</i>	Error: Internal Server Error
	Response body
	<pre>Internal Server Error</pre>
	Response headers
	<pre>content-length: 21 content-type: text/plain; charset=utf-8 date: Fri, 03 Jul 2026 12:19:36 GMT server: uvicorn</pre>

GET:

Responses	
Curl	
<pre>curl -X 'GET' \ 'http://127.0.0.1:8000/caronas/?skip=0&limit=100' \ -H 'accept: application/json'</pre>	
Request URL	
<pre>http://127.0.0.1:8000/caronas/?skip=0&limit=100</pre>	
Server response	
Code	Details
200	Response body <pre>[{ "cpf_motorista": "00000000011", "placa_veiculo": "ABC1A11", "id_origem": 4, "id_destino": 1, "data": "2026-07-12", "horario": "18:00:00", "vagas_disponiveis": 3, "id_carona": 32 }, { "cpf_motorista": "00000000011", "placa_veiculo": "ABC1A11", "id_origem": 1, "id_destino": 5, "data": "2026-07-13", "horario": "06:30:00", "vagas_disponiveis": 2, "id_carona": 33 }]</pre>

GET por ID:

Responses	
Curl	
<pre>curl -X 'GET' \ 'http://127.0.0.1:8000/caronas/890' \ -H 'accept: application/json'</pre>	
Request URL	
<pre>http://127.0.0.1:8000/caronas/890</pre>	
Server response	
Code	Details
200	Response body <pre>{ "cpf_motorista": "00000000011", "placa_veiculo": "ABC1A11", "id_origem": 2, "id_destino": 1, "data": "2026-07-03", "horario": "09:46:51.751000", "vagas_disponiveis": 2, "id_carona": 890 }</pre>

PUT:

Responses

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/caronas/1' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "vagas_disponiveis": 0
  }'
```

Request URL

http://127.0.0.1:8000/caronas/1

Server response

Code	Details
500 <i>Undocumented</i>	Error: Internal Server Error

Response body

Internal Server Error

DELETE:

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/caronas/890' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/caronas/890

Server response

Code	Details
500 <i>Undocumented</i>	Error: Internal Server Error

Response body

Internal Server Error

Response headers

```
content-length: 21
content-type: text/plain; charset=utf-8
date: Fri, 03 Jul 2026 13:53:10 GMT
server: uvicorn
```

6. Integração com a Base de Dados

A integração entre a aplicação FastAPI e o banco de dados PostgreSQL foi estabelecida por meio de bibliotecas de conexão nativas do ecossistema Python, garantindo a comunicação direta e o desacoplamento das consultas complexas do SGBD na camada de código. O arquivo `database.py` é o responsável por gerenciar dinamicamente essa comunicação, mantendo a abertura e o fechamento de sessões com o banco de dados real utilizando as credenciais criadas na etapa DCL. O arquivo `models.py` espelha as restrições lógicas do modelo relacional, mapeando especificamente a entidade CARONA (alvo do desenvolvimento desta etapa) em objetos manipuláveis pelo código em Python.

Para testar a robustez do esquema relacional e a consistência das diretivas de segurança, as operações de CRUD da rota `/caronas/` foram executadas e validadas sob duas perspectivas de conexões intercambiáveis diretamente no Swagger UI (`/docs`):

1. **Acesso Administrativo (`unicarona_admin`):** Quando a aplicação se conecta sob esse papel, todas as operações CRUD funcionam livremente. Requisições `POST /caronas/`, `PUT /caronas/{id_carona}` e `DELETE /caronas/{id_carona}` retornam com sucesso o status

HTTP 200 OK, persistindo as alterações de estado fisicamente no banco (pode-se verificar esse comportamento nas imagens).

2. **Acesso Restrito (unicarona_leitura):** Sob esse perfil, a política de privilégio mínimo é forçada pelo PostgreSQL. O usuário consegue realizar normalmente as consultas GET /caronas/ e GET /caronas/{id}. Contudo, ao disparar requisições de escrita, o SGBD barra a instrução internamente e lança uma exceção de violação de privilégio (permission denied for table carona). Como a camada lógica da API recebe esse erro e não implementa uma tradução explícita de exceção de banco de dados para erro HTTP, a plataforma encerra a execução com o status 500 Internal Server Error, provando fisicamente a barreira e a eficácia das travas DCL configuradas no banco (pode-se verificar esse comportamento nas imagens).

7. Anexos

- [unicarona_entrega4.sql](#) → Comandos SQL das visões, criação de usuários e permissões.
- [SLIDE Unicarona \(Apresentação\).pdf](#) → Link para acesso do slide.
- [printscreen_app/](#) → Pasta com imagens do sistema funcionando.
- [codigo_app/](#) → Código-fonte da aplicação (em Python, JS, etc.).
- https://github.com/PabloBr4ndao/Fundamentos_BancoDeDados/tree/main/Projeto_Unicarona → Repositório do projeto no GitHub.